# C Programming

NOTE BOOK

Name : _____

Class : _____

College Name : _____


## Author By :

Er. Ninama Kuldip N.

Er. Devi Amit T.

Contact No.
+91 97253 09343
+91 87349 67027

## Introduction to Programming

**Q.1 what is c language ? full explanation.**

- **C is a middle level procedural oriented programming language.**
- **It is developed by "Dennis Ritchie" at AT & T 'S bell lab in the year 1972 in USA.**
- **Syntax of C language program.**

```
#include<stdio.h>
main()
{
        //BLOCK OF CODE
}
```

**Q.2 How to write C program and steps ?**

1. **Documentation ( comment // or /*…….*/)**
2. **Preprocessor directive (#include)**
3. **Global declaration.**
4. **Function declaration.**

**Note :**

- **C is a Case-sensitive language.**
- **.c is an extension of c language.**
- **C language supports 32 keywords.**

```
#include<stdio.h>
#include<conio.h>
void main()
{
        printf("Hello world!");
        getch();
}

Output :
Hello world!
```

**Q.3 Advantages & Disadvantages of C language.**

- **Advantages of c language**
  - **C language is a open source.**
  - **C language is a easy to learn and understanding language.**
  - **C is a portable.**
  - **C is a easy debugging.**
  - **Dynamic memory allocation.**

- **Disadvantages of c language**
  - ❖ **C is not approaches to object oriented because it is procedural oriented.**
  - ❖ **C is exception handling.**
  - ❖ **C has not been garbage collector.**
  - ❖ **C is a platform dependent.**

**Q.4 what is keyword ? Full explanation.**

- ❖ **Keyword is a reserved word whose meaning already defined in the compiler.**
- ❖ **A programmer meaning of keyword can not be changed.**
- ❖ **All the keywords must be written in lowercase.**

| int | extern | For | sizeof |
|---|---|---|---|
| char | goto | const | short |
| float | break | If | struct |
| double | continue | Else | union |
| void | do | Enum | typedef |
| auto | switch | signed | volatile |
| register | case | unsigned | defult |
| return | while | Static | long |

**Example :**

**int a =10;**

```
// keyword Example of program

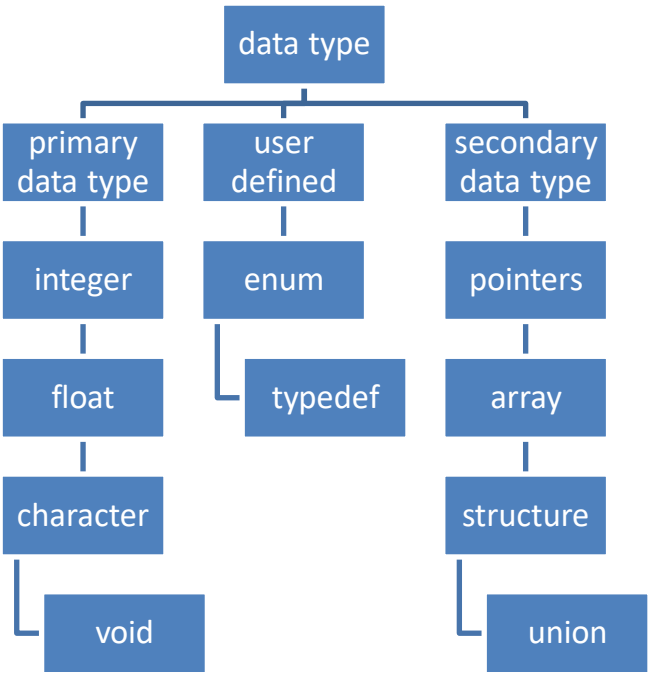
#include<stdio.h>
#include<conio.h>
void main()
{
        int a;
        printf("enter the value of a: ");
        scanf("%d",&a);
        printf("You are entered integer:%d",a);
        getch();
}

Output :
You are entered integer:10
```

**Q.5 what is data type ? full explanation.**

- ❖ **Data type :**

- ✓ **Data type specifies the size & type of values that can be stored in an variable.**
- ✓ **C has various types of data type.**




**Example:**

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int a;
        clrscr();
        printf("%d", sizeof(a));
        getch();
 }

output:
 2
```

**Q.6 what is variable? Full explanation**

- ❖ **Variable:**
  - ✓ **Variable is the name of memory location where we store data.**

- ✓ **Variables are changeable we can change the value of variable during the execution of program and variable can be refused many times.**

- ❖ **There are many types of variable**
  1. **Local variable**
  2. **Global variable**
  3. **Static variable**
  4. **Automatic variable**
  5. **Extern variable**
- ❖ **Rules for define variable:**
- ❖ **The following are the rules for writing a variable name in a program in c.**
  - ✓ **The first character may be an alphabet an underscore( _ )**
  - ✓ **The first character of variable name can't be a digit.**
  - ✓ **Blank (white spaces) are not allowed.**
  - ✓ **Special characters such as arithmetic operators, # , ^ ,can't be used.**
  - ✓ **Reserved c words can't be used as variable names.**
  - ✓ **The maximum length of a variable name is up to 31 characters how ever the maximum length depends upon the compiler.**

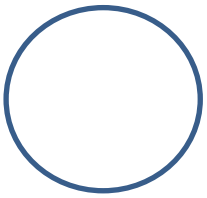
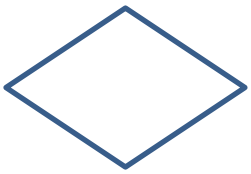
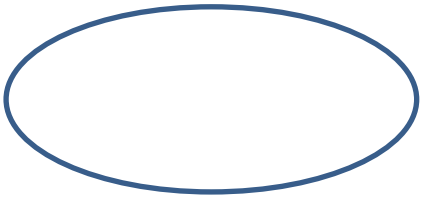
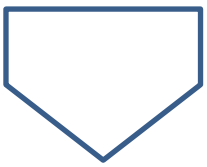
**Example:**

```
#include<stdio.h>
#include<conio.h>
 void main()
 {
    int a =10;
    printf ("%d",a);
   getch();
  }
```

**Output:**
**10**

**Q.7 Explain flowchart ? full details.**

- ❖ **Flowchart :**
  - ✓ **Flowchart is graphical or pictorial representation of process it's called a flowchart**
  - ✓ **A flowchart symbols represent the are as under:**

| | |
|---|---|
|  | **Connector** |
|  | **Process** |
|  | **Condition / Decision** |
|  | **Input / Output** |
|  | **Terminal** |
|  | **Flow Lines** |
|  | **Off Page Connector** |

- **Limitations of flowchart explain are as under.**

  - ✓ **It is difficult to draw a flowchart for a complex problems for example subroutines, function, loops and file handling.**
  - ✓ **If some changes are required within a flowchart then the flowchart is to be redrawn.**
  - ✓ **If there is any error in the algorithm then flowchart would also be wrong.**

- **Advantages of flowchart:**

  - ✓ **It helps to understand logic of given problem.**
  - ✓ **Problems can be analyzed in a more effective.**
  - ✓ **The flowchart helps in debugging process.**
  - ✓ **Easy to identify the mistakes done by any level.**
  - ✓ **Flowchart act as a guide or blueprint during the system analysis and program development phase.**
  - ✓ **Easy to branching and looping.**

- **Disadvantages of flowchart:**

  - ✓ **Difficult to use and manage.**
  - ✓ **Time consuming.**
  - ✓ **Difficult for modification.**
  - ✓ **People need to understand the meaning of symbol.**

**Q.8 Explain Algorithm ? full details.**

- **An Algorithm is a step by step problem solving method.**
- **Algorithm is used to write simple English language.**
- **It is less time consuming.**
- **Easy to modify**
- **In algorithm easy to debug error.**

- **Advantages of Algorithm :**

  - ✓ **Algorithm is a step by step process , so it is very easy to understand.**
  - ✓ **Very easy to write (it does not require any programming ability).**
  - ✓ **An algorithm act as a blueprint of a program.**
  - ✓ **It is easy to debug because algorithm has its own logical sequence.**

- **Disadvantages of algorithm :**

  - ✓ **It is time consuming.**
  - ✓ **Difficult to understand large and complex problems.**
  - ✓ **Difficult to show branching and looping.**

**Q.9 what is identifier ? full explanation ?**

- **Identifier :**
  - ✓ **Identifier refers to the name that is used to identify variables , functions and so on**

    **Or**
  - ✓ **Identifier names you supply for variable types , functions and labels in your program it's called identifier.**
  - ✓ **Example :**
    - **int salary ;**
    - **double money ;**
    - **void sum( )**

- **Rules of identifier :**
  - ✓ **First character must be an alphabet or underscore ( _ ).**
  - ✓ **Identifier names consists of only letters , digits and underscore (_).**
  - ✓ **An identifier name should have less than 31 characters.**
  - ✓ **Any standard c language keyword can not be used as a variable.**
  - ✓ **An identifier should not contain a space.**

**Q.10 what is operator ? full explanation.**

- **Operator :**
  - ✓ **Operator is a symbol that tells the compiler to perform mathematical and logical task.**
  - ✓ **Types of operator :-**
    - **Arithmetic operator**
    - **Relational operator**
    - **Logical operator**
    - **Assignment operator**
    - **Conditional operator**
    - **Special operator**
    - **Bitwise operator**

**Q.11 Explain arithmetic operator.**

- **Arithmetic operators are the type of operators used to perform basic math operations.**
- **Like addition , subtraction , division and multiplication.**

| Sr.no | Operator | Operator name | Expressions |
|---|---|---|---|
| 1 | + | Addition | A + B |
| 2 | - | Subtraction | A - B |
| 3 | * | Multiplication | A * B |
| 4 | / | Division | A / B |
| 5 | % | Modulo | A % B |

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int a=10 , b=20 , c;
        clrscr();

        c = a + b;
        c = a – b;
        c = a * b;
        c = a / b;
        c = a % b;

        printf("a+b is :%d\n",c);
        printf("a-b is :%d\n",c);
        printf("a*b is :%d\n",c);
        printf("a/b is :%d\n",c);
        printf("a%b is :%d\n",c);

        getch();
}

Output :
30
-10
200
0
2
```

**Q.12 Explain Relational Operator.**

- **Relational operators defines some kinds of relations between two operands.**
- **It check numerical equally or non-equally condition.**

| Sr.no | Operator | Operator name | Expression |
|---|---|---|---|
| 1 | == | Equals to | a==b |
| 2 | != | Not equals to | a!=b |
| 3 | < | Less than | a<b |
| 4 | > | Greater than | a>b |
| 5 | <= | Less than equal to | a<=b |
| 6 | >= | Greater than equal to | a>=b |

```
#include<stdio.h>
#include<conio.h>
void main()
{
         int a = 10;
        if(a%2==0)
        {
                printf("%d is even number",a);
        }
        getch();

}
```

**Q.13 Explain Logical Operator.**

- **Logical operators in c are used to combine multiple conditions.**
- **Logical operators returns either 0 or 1.**
- **it depends on whether the expression result is true or false.**
- **We have 3 logical operators in the c language.**
    1. **Logical AND (&&)**
    2. **Logical OR (||)**
    3. **Logical NOT (!)**

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int a=10, b=20;
        if(a>0 && b>0)
        {
                printf ("both are largest ");
        }
        else
        {
                printf("both are smallest");
        }
        getch();
}
```

**Output :**

**both are largest**

**Q.14 Explain Assignment Operator.**

- **Assignment operator are used to something to assign the variable or value.**
- **We have two types of assignment operator.**
    - ✓ **Simple assignment (=)**
    - ✓ **Compound assignment (+= , -= , *= , /= , %= , <<= , >>=)**
- **Example**
    - ✓ **int a = 10;**
    - ✓ **int a+= 10;**
    - ✓ **int a = a+10;**
- **syntax**
    - ✓ **data-type variable = value;**
    - ✓ **int a = 10;**

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int a=10;
        a+=10;
        printf("Assign operator:%d",a);
        getch();
}

Output :
Assign operator : 20
```

**Q.15 Explain conditional operator or Ternary Operator.**

- **Conditional operator is a ternary operator because it works on three operands.**
- **The conditional operator behaves like an if-else statement.**
- **Syntax**
  - ✓ **Condition ? value if-true : value if-false;**

```
//write a c program to find greatest among three numbers.
#include<stdio.h>
#include<conio.h>
void main()
{
        int a , b , c , max;

        printf("Enter three Numbers:");
        scanf("%d %d %d", &a , &b , &c);

        max = (a>b)?(a>c ? a:c) : (b>c ? b:c);

        printf("the greatest number:%d" , max);
        getch();
}

Output :
the greatest number:30
```

**Q.16 Explain Bitwise operator.**

- **Bitwise operators are used to perform individual bits of a number.**
- **It can be used only integer type values not float , double etc.**

| Operators | Meaning | A | B |
|---|---|---|---|
| & | Bitwise AND | 0 | 0 |
| \| | Bitwise OR | 1 | 1 |
| ^ | Bitwise XOR | 0 | 0 |
| ~ | Complement | 0 | 1 |
| << | Left shift | | |
| >> | Right shift | | |

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int a=4 , b=5;
        clrscr();

        printf("Bitwise  AND %d\n", a & b);
        printf("Bitwise OR %d\n", a | b);
        printf("Bitwise XOR %d\n", a * b);
        getch();
}

Output :
Bitwise AND 4
Bitwise OR 5
Bitwise XOR 1
```

**Q.17 what is if statement ? full explanation.**

- **If statement is used when we want to test a condition.**
- **Syntax :**

```
if(condition)
{
        //block of code
}
```

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int a = 10;
        if(a>0)
        {
                printf("%d is greater than zero",a);
        }
        getch();

}

output:
10 is greater than zero.
```

**Q.18 what if else statement ? full explanation.**

- **It is used to execute two statements either if statement or else statement for a single condition.**
- **syntax:**

```
if (condition)
{
        //statement 1;
}
else
{
        //statement 2;
}
```

- **Algorithm:**

**Step 1: start**
**Step 2 : input age**
**Step 3 : if (age greater 18) then print (“vote”) otherwise print (“not”)**
**Step 4 : end.**

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int age;
        clrscr();
        printf (“enter age…\n”)
        scanf (“%d, &age);
        if (age>=18)

        {
                printf (“eligible for vote”);
        }
        else
        {
                printf (“not eligible for vote”);
        }
        getch ();
        }
```

**Output:**
**enter age…**
**18**
**eligible for vote**

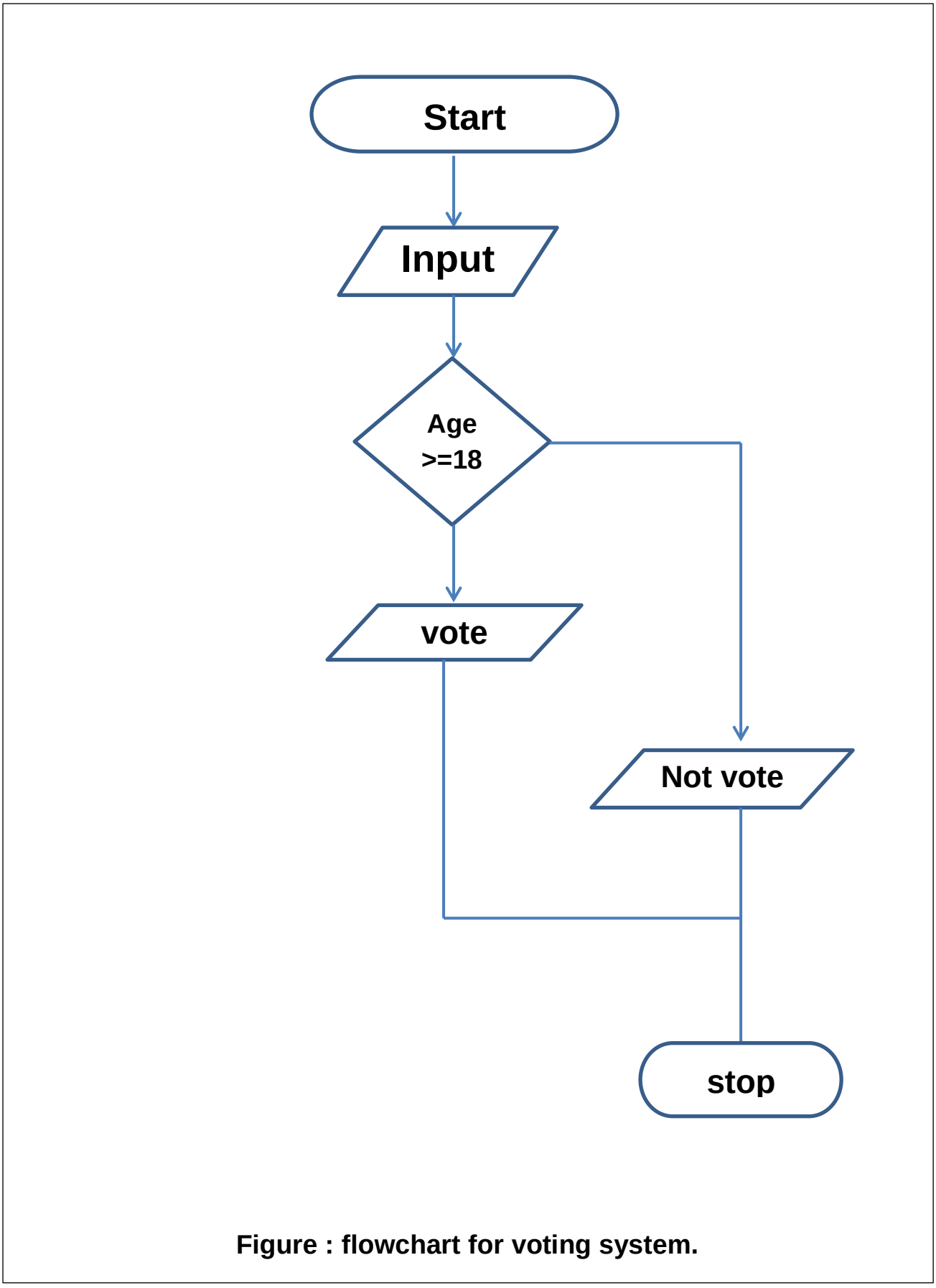



**Figure : flowchart for voting system.**

**Q.19 what is if else-if ladder statement ?**

- **if-else if statement used when we have multiple (more than two) condition.**
- **Syntax :**

```
if(condition)
{
        statement 1;
}
else if(condition)
{
        statement 2;
}
else
{
        statement 3;
}
```

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int marks;
        clrscr();
        printf("enter marks…");
        scanf("%d",&marks);

        if(marks == 100)
        {
                printf("genius student");
        }
        else if(marks>=80 && marks<100)
        {
                printf("topper student");
        }
        else if(marks>=60 && marks<=79)
        {
                printf("first division");
        }
        else
        {
                printf("average student");
        }
        getch();
}
```

**Output :**
**enter marks… 100**
**genius student**

**Q.20 what is nested if else statement ?**

- **Whenever we define if else block inside another if else block called nested if else statement.**
- **Syntax :**

```
if(condition 1)
{
        if(condition 2)
        {
                statement 1;
        }
        else
        {
                statement 2;
        }
}
else
{
        if(condition 3)
        {
                statement 3;
        }
        else
        {
                statement 4;
        }
}
```

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int a=10 , b=20 , c=30;
        clrscr();
        if(a>b)
        {
                if(a>c)
                {
                        printf("%d ",a);
                }
                else
                {
                        printf("%d",c);
                }
        }
```

```
else
{
        if(b>c)
        {
                printf("%d",b);
        }
        else
        {
                printf("%d",c);
        }
}

        getch();

}
```

**Output :**

**Q.21 what is switch statement ? full explanation.**

- **Switch statement used when we want to select any one case out of multiple cases.**
- **Syntax :**

```
switch(exp)
{
        case 1:
                statement;
                break;
        case 2:
                statement;
                break;
        case n:
                statement;
                break;
        default:
                statement;
                break;
}
```

```
#include<stdio.h>
#include<conio.h>
void main()
{
	int a,b,c;
	int ch;

	printf("enter value num1:");
	scanf("%d",&a);

	printf("enter value num2:");
	scanf("%d",&b);

	printf("enter choice:");
	scanf("%d",&ch);

	switch(ch)
	{
		case 1:
			c=a+b;
			printf("%d",c);
			break;
		case 2:
			c=a-b;
			printf("%d",c);
			break;
		case 3:
			c=a*b;
			printf("%d",c);
			break;
		case 4:
			c=a/b;
			printf("%d",c);
			break;
		case 5:
			c=a%b;
			printf("%d",c);
			break;
		default:
			printf("invalid choice");
			break;
	}
	getch();
}
```

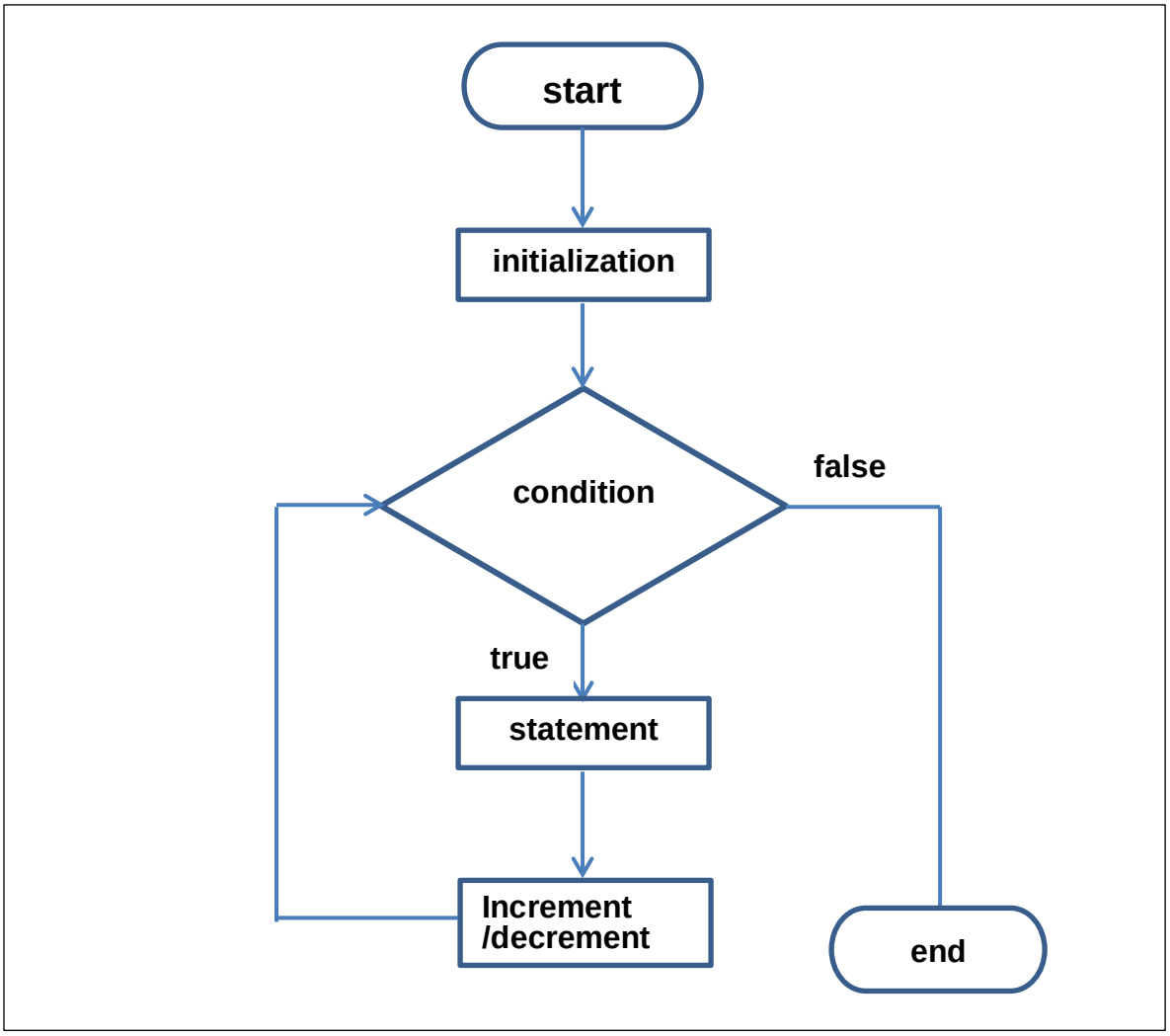
**Q.22 what is loop ? full explanation.**

- **Loop is used to repeat block of code until the given condition is true.**
- **Types of loops :**
  1. **For loop**
  2. **While loop**
  3. **Do while loop**

**Q.23 Explain for loop ? full explanation**

- **For loop as a way to well a computer, Do this specific task a certain number of times.**
- **It's a shortcut that saves you from writing the same line of code over and over again.**
- **Syntax:**

```
for(initialization; condition; increment/decrement);
{
        statement;
}
```



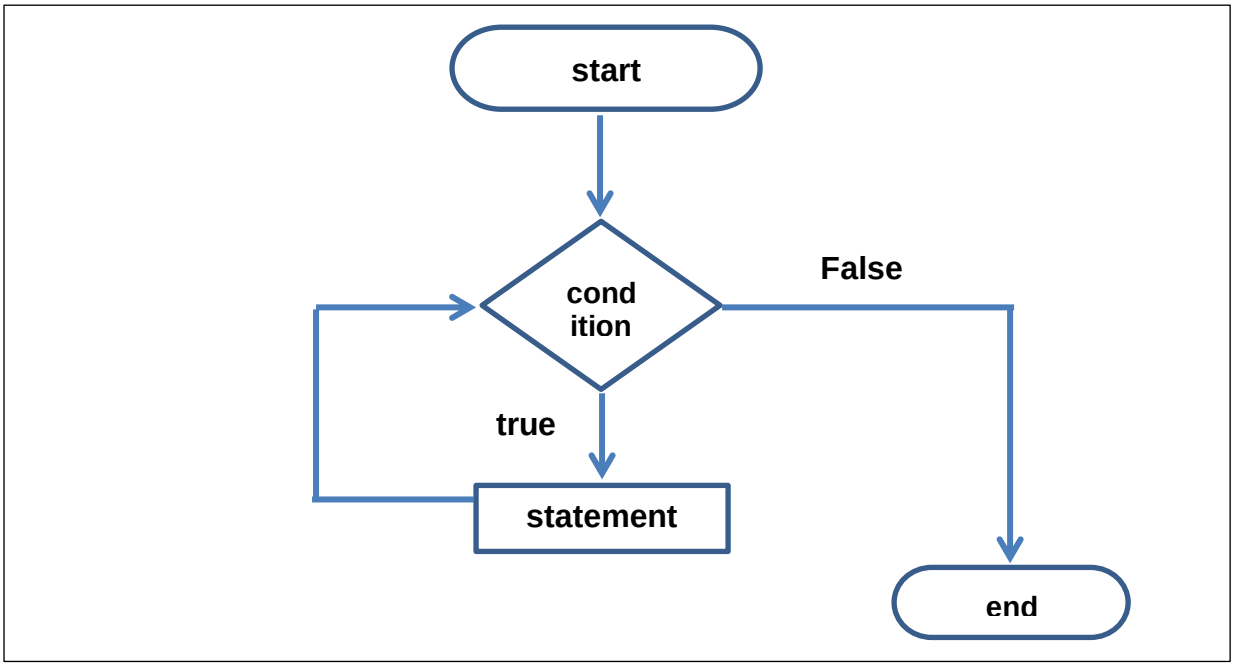
```
#include<stdio.h>
#include<conio.h>
void main()
{
        int i;
        clrscr();
        for(i=1; i<=10; i++)
        {
                printf("%d\t",i);
        }
        getch();
}

Output :
1 2 3 4 5 6 7 8 9 10
```

**Q.24 explain while loop ? full explanation.**

- A while loop in c programming is a control flow statement that allows code to be executed repeatedly as long as a specified condition remains true.
- It is classified as an entry-controlled loop because it tests the condition before executing the loop body.
- Syntax :

```
While(condition)
{
        //statement;
}
```



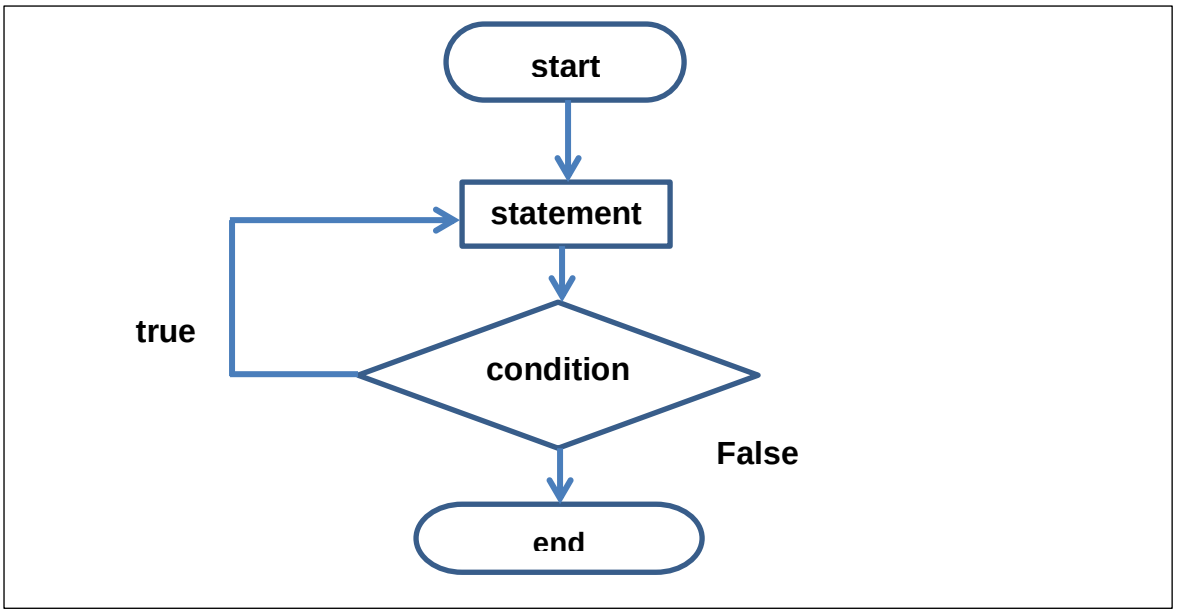
```
#include<stdio.h>
#include<conio.h>
void main()
{
        int i=1;
        clrscr();
        while(i<=10)
        {
                printf("%d\t",i);
                i++;
        }
        getch();
}

Output :
1 2 3 4 5 6 7 8 9 10
```

**Q.25 explain do while loop.**

- A do-while loop in c programming is a control structure that executes a block of code at least once before checking a condition.
- The condition is evaluated at the end, it is known as exit-controlled or post-tested loop.
- Syntax :

```
do
{
        statement;
}while(condition);
```



```
#include<stdio.h>
#include<conio.h>
void main()
{
        int i=1;
        clrscr();
        do
        {
                printf("%d\t",i);
                i++;
        }while(i<=10);
        getch();
}
```

**Output :**
**1 2 3 4 5 6 7 8 9 10**

**Q.26 what is nested loop ?**

- **A loop which contain inside another loop called nested loop.**
- **Syntax :**

```
for(initialization; condition; increment/decrement)
{
        for(initialization; condition; increment/decrement)
        {
                statement;
        }
}
```

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int i , j;
        clrscr();

        for(i=1;  i<=5;  i++)
        {
                for(j=1;  j<=i;  j++)
                {
                        printf(" * ");
                }
                printf("\n");
        }
        getch();
}
```

**Q.27 what is jumping statement ?**

- **A jump statement in C is a control structure used to unconditionally transfer program execution from one point to another within the same function.**
- **They are primarily used to alter the standard sequential flow, handle errors, or manage complex loops and switch-case blocks.**
- **Types of jumping statement :**
  - ✓ **There are five types of jumping statement.**
    1. **break**
    2. **continue**
    3. **exit**
    4. **goto**
    5. **return**

**Q.28 what is break statement with suitable example.**

- **In the C programming language, the break statement is a jump control statement that immediately terminates the execution of the nearest enclosing loop or switch statement.**
- **Core Functions:**
  - ✓ **Loop Termination:**
    - **When encountered inside a for, while, or do-while loop, it forces the loop to end instantly, even if the loop's original condition is still true. Control then moves to the statement immediately following the loop.**
  - ✓ **Switch Case Management:**
    - **In a switch statement, break prevents "fall-through" by exiting the switch block once a matching case is executed. Without it, the program would continue executing all subsequent cases.**
- **Syntax:**
  - ✓ **It consists of the keyword break followed by a semicolon: break;**
- **Placement:**
  - ✓ **It can only be used inside the body of a loop or a switch statement. Using it anywhere else (like inside a simple if block that is block that is not within a loop) will cause a compilation error.**

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int i;
        for(i=1; i<=10; i++)
        {
                if(i==5)
                {
                        break;
                        printf("%d", i);
                }
        }
        printf("N . K . Education hub");
        getch();
}
```

**Output :**

**Q.29 what is continue statement with suitable example.**

- **In C programming, the continue statement is a jump statement used to skip the current iteration of a loop and immediately move to the next one.**
- **It allows you to bypass specific lines of code within a loop body based on a condition without terminating the loop entirely.**
- **Key Characteristics**
  - ✓ **Loop Only: It can only be used inside loop constructs like for, while, and do-while. Using it outside a loop (e.g., in a switch statement or main function body) will cause a compilation error.**
  - ✓ **Jump Behavior: When encountered, the program control jumps to the end of the loop body and proceeds to the loop's update expression or condition check.**
  - ✓ **Scope: In nested loops, continue only affects the innermost loop that contains it.**

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int i;
        for(i=1; i<=10; i++)
        {
                if(i==7)
                {
                        continue;
                        printf("%d", i);
                }
                printf("N K Education Hub");
        }
        getch();
}
```

**Output:**

**Q.30 what is exit statement with suitable example.**

- **The exit() function in C is used to immediately terminate the execution of the entire program and return control to the operating system.**
- **Unlike the return statement, which only exits the current function, exit() stops the program from any point it is called.**
- **Syntax and Usage**
  - ✓ **To use the exit() function, you must include the <stdlib.h> header file.**
- **Status: An integer value passed back to the parent process or OS.**
  - ✓ **0 or EXIT_SUCCESS: Indicates the program completed successfully.**
  - ✓ **1 (or non-zero) or EXIT_FAILURE: Indicates the program terminated due to an error.**
- **Other Termination Functions**
  - ✓ **Exit(): Terminates the program immediately without performing cleanup tasks like flushing buffers or calling at exit functions.**
  - ✓ **abort(): Causes abnormal program termination. It does not perform cleanup and may generate a core dump for debugging.**
  - ✓ **assert(): A macro used for debugging that calls abort() if a specific condition is false.**

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
void main()
{
        int i;
        for(i=1; i<=10; i++)
        {
                if(i==7)
                {
                        exit(0);
                        printf("%d", i);
                }
        }
        printf("\namit devi");
        getch();
}

output :
```

**Q.31 what is goto statement with suitable example.**

- **The goto statement in c is an unconditional jump statement that transfers the program's control flow to a specified label within the same function.**
- **Syntax and Structure**
  - ✓ **A goto statement requires two parts: the goto keyword followed by a label name, and the label itself (an identifier followed by a colon).**

```
goto label_name;

...

label_name:

// statements to execute after the jump
```

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int i=1;
        label:
        printf("%d", i);
        i++;

        if(i<=10)
        {
                goto label;
        }
        getch();
}
```

**Output :**

**Q.32 what is return statement with suitable example.**

- **The return statement in C programming is used to terminate a function's execution and pass control (and optionally a result) back to the calling function.**
- **Key Functions of the Return Statement**
  - ✓ **Ends Execution: When a return is encountered, the function stops immediately, even if there is remaining code in its block.**
  - ✓ **Transfers Control: It shifts the program's focus back to the point where the function was originally called.**
  - ✓ **Passes Values: For functions with a specific return type, the statement transmits a single value (integer, character, pointer, etc.) back to the caller.**
- **The syntax varies depending on whether the function is meant to return a value:**
- **Functions with a Return Value:**
  - ✓ **return expression;**
  - ✓ **The "expression" can be a constant, variable, or calculation.**
  - ✓ **The data type must match the function's declared return type (e.g., int, float, char).**

- **Void Functions (No Return Value):**
  - ✓ **return;**
  - ✓ **Used in functions with a void return type.**
  - ✓ **It is optional at the end of the function but can be used to exit the function early if a certain condition is met.**
- **Usage in the main() Function**
  - ✓ **In the main() function , return 0; is conventionally used to indicate successful program termination to the Operating System.**
  - ✓ **Return 0: Signals success.**
  - ✓ **Non-zero value: Often signals an error or abnormal termination.**

```
#include<stdio.h>
#include<conio.h>
        // void method without return statement
void print()
{
   printf("Welcome to N K Education Hub");
}

int main()
{
        // Calling print
   print();
   return 0;
}

Output :
```

**Q.33 what is array ? full explanation.**

- **Array is a collection of similar type of data items in a contiguous memory location.**

**Or**

- **Array is a fix size of sequence collection of elements of similar data type is called array.**
- **Array index always start from zero (0).**
- **Syntax :**

```
datatype array-name[size];

int a[10];
```

- **We have two types of array**
    1. **1D (one dimensional) Array**
    2. **2D (two dimensional) Array**

**Q.34 Explain 1D Array with suitable example.**

- **A one-dimensional (1D) array in C is a collection of elements of the same data type stored in contiguous (back-to-back) memory locations.**
- **It acts as a single variable that can hold multiple values, each accessible via a unique index.**

| **Key Characteristics** | |
|---|---|
| **Homogeneous :** | **All elements must be of the same type (e.g., all int or all char)** |
| **Zero-indexed :** | **The first element is at index 0, and the last is at index size – 1.** |
| **Fixed Size :** | **Standard arrays have a fixed size determined at compile-time that cannot be changed during execution.** |
| **Contiguous memory :** | **Elements are stored sequentially, making access extremely fast (0(1) time complexity).** |

- **Syntax and examples.**
    - ✓ **You can declare, initialize, and access 1D arrays using these standard patterns:**

| Action | Syntax/example |
|---|---|
| **Declaration** | **datatype array_name[array_size];** |
| **Full Initialization** | **int arr[3] = {10, 20, 30};** |
| **Partial Initialization** | **int arr[5] = {1, 2}; (rest are set to 0)** |
| **Auto-Sizing** | **int arr[ ] = {5, 10, 15}; (compiler determines size as 3)** |
| **Access/Update** | **arr[1] = 50; (changes the second element to 50)** |

```
//write a c program to show the example of 1D Array.
#include<stdio.h>
#include<conio.h>
void main()
{
        int a[5] = {70,90,35,65,87};
        int i;
        clrscr();

        for(i=0; i<5; i++)
        {
                printf("%d",a[i]);
        }
        getch();
}

Output :
70 90 35 65 87
```

**Q.35 Explain 2D Array with suitable example.**

- **a two-dimensional (2D) array is essentially an "array of arrays."**
- **It organizes data into a tabular format consisting of rows and columns, similar to a matrix or a spreadsheet.**
- **To declare a 2D array, specify the data type, followed by the array name and two sets of square brackets representing the number of rows and columns.**
- **Key Characteristics**
  - ✓ **Contiguous Memory: All elements occupy a single, continuous block of memory for efficient access.**
  - ✓ **Total Elements: Calculated as rows * columns. For int matrix[3][4], it stores 12 integers.**
  - ✓ **Applications: Commonly used for representing mathematical matrices, game boards (like Tic-Tac-Toe), and image pixel data.**

```
//write a c program to show the example of 2d array
#include<stdio.h>
#include<conio.h>
void main()
{
        int a[2][2] , i , j;
        printf("enter array elements:");

        for(i=0; i<2; i++)
        {
                for(j=0; j<2; j++)
                {
                        scanf("%d",&a[ i ][ j ]);
                }
        }

        printf("matrix elements\n");

        for(i=0; i<2; i++)
        {
                for(j=0; j<2; j++)
                {
                        printf("%d",a[ i ][ j ]);
                }
                printf("\n");
        }
        getch();|
}
```

**Output :**

```
Enter Array Elements :
10
20
30
40

Matrix Elements :
10    20
30    40
```

**Q.36 what is string ? full explanation.**

- **String is a sequence of characters and it always ends with null ("\0") character.**
- **Syntax :**

  **Datatype var-name[ ] = "string";**

  **char a[10] = "N K EDUCATION HUB";**

| Sr.no | String operation name | Declaration |
|---|---|---|
| 1 | strlen() | String length |
| 2 | strcpy() | String copy |
| 3 | strcat() | String concate(merge) |
| 4 | strcmp() | String compare |
| 5 | strlwr() | String lowercase |
| 6 | strupr() | String uppercase |

- **String input / output :**
  1. **gets() :- input string**
  2. **puts() :- print string**

```
//reading strings from terminal and writing strings to screen program in c language.

#include<stdio.h>
#include<conio.h>
void main()
{
        char str[50];
        puts("Enter Any String:")
        gets(str);

        puts(str);
        getch();
}

Output :
Enter Any String: N K EDUCATION HUB
N KEDUCATION HUB
```

**Q.37 write a c program to perform string operation lowercase to uppercase.**

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
        char str[50];
        puts("Enter Any String:");
        gets(str);

        puts(strupr(str));

        getch();
}

Output :
Enter Any String:
amit
AMIT
```

**Q.38 write a c program to perform string operation uppercase to lowercase.**

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
        char str[50];
        puts("Enter Any String:");
        gets(str);

        puts(strlwr(str));

        getch();
}

Output :
Enter Any String:
AMIT
amit
```

**Q.39 write a c program to perform string operation string reserve.**

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
        char str[50];
        puts("Enter Any String:");
        gets(str);

        puts(strrev(str));

        getch();
}

Output :
Enter Any String:
Kuldip Ninama
Amanin Pidluk
```

**Q.40 write a c program to perform string operation string length.**

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
        char str[50];
        puts("Enter Any String:");
        gets(str);

        printf("%d",strlen(str));

        getch();
}

Output :
Enter Any String:
Kuldip Ninama
13
```

**Q.41 write a c program to perform string operation string copy.**

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
        char str[50] , str2[40];

        puts("Enter Any String:");
        gets(str);

        strcpy(str2,str);

        puts(str2);
        getch();
}

Output :
Enter Any String:
Kuldip Ninama
Kuldip Ninama
```

**Q.42 write a c program to perform string operation string compare.**

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
        char str[50] , str2[40];
        puts("Enter Any String:");
        gets(str);
        puts("Enter Any String:");
        gets(str2);
                if(strcmp(str,str2)==0)
                {
                        puts("Both are same");
                }
                else
                {
                        puts("Both are not same");
                }
        getch();
}
```

**Q.43 write a c program to perform string operation string concatenation.**

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
        char str[50] , str2[40];

        puts("Enter Any String:");
        gets(str);

        puts("Enter Any String:");
        gets(str2);

        strcat("str,str2");
        puts("str");

        getch();
}
```

**Output :**

```
Enter Any String : Amit
Enter Any String : Devi
Amit Devi
```

**Q.44 what is pointer ? full explanation.**

- **Pointer is a variable that can able to hold address of another variable.**
- **While working with pointer we need to required two unary operator. i.e.**
  1. **& : address of operator**
  2. *** : value at address operator.**
- **Syntax :**

  **datatype *var-name;**
- **Example :**
  - ✓ **int a = 10 , *p , **q;**
  - ✓ **p = &a;**
  - ✓ **q = &p;**

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int *p , **q , a=10;
        p = &a;
        q = &p;

        printf("%d",a);      //10
        printf("%d\n",&a); //address a
        printf("%d\n",p); //address a
        printf("%d\n",*p); //10
        printf("%d\n",q); //address p
        printf("%d\n",*q); //address a
        printf("%d\n",**q); //10

        getch();
}

Output :
10
6487564
6487564
10
6487568
6487564
10
```

- types of pointer :
  - ✓ void pointer
  - ✓ wild pointer
  - ✓ null pointer
  - ✓ dangling pointer
  - ✓ pointer to pointer
  - ✓ function pointer

**Q.45 what is function ? full explanation.**

- Function is a block of code which take input from the user, processed it and produce output.
- Function run only when it is called.
- We have two types of functions.
  - ✓ User-define function
  - ✓ Pre-define function

| Types of function | |
|---|---|
| **User-define function** | **Pre-defined function** |
| void add(); | printf(); |
| int sum(); | scanf(); |
| void main(); | strcpy(); |
| void bark(); | puts(); |

**User define function :**

- **A user-defined function is a custom block of code created by a programmer to perform a specific task.**
- **Unlike built-in library functions like printf() or scanf(), these functions are defined within your program to improve modularity, readability, and code reusability.**

**Syntax :**

```
return-type function-name()
{
        //statements;
}
```

```
#include<stdio.h>
#include<conio.h>
int sum();     //user-defined function
{
        int a , b;
        printf("Enter Value:");   //pre-defined
        scanf("%d %d",&a,&b);
        return a+b;
}
int main()
{
        int store = sum();
        printf("%d",store);
        return 0;
}

Output :
Enter Value : 100 200
300
```

**Q.46 what is storage class ? full explanation.**

- **Storage class defines the scope and lifetime of variables and functions.**
- **Types of storage class**
  - ✓ **auto**
  - ✓ **static**
  - ✓ **extern**
  - ✓ **register**

| Storage class | Memory | Default value | Scope | Life-time |
|---|---|---|---|---|
| auto | RAM | Garbage | Within block | Still block is a active |
| static | RAM | 0 | Within block | Till the termination of program |
| extern | RAM | 0 | Anywhere | Till the termination of program |
| register | Register | Garbage | Within block | Still the block is active |

```
//write a c program using auto variable and printing a value of a & b.
#include<stdio.h>
#include<conio.h>
int main()
{
        auto int a;
        if(1==1)
        {
                auto int b=20;
                printf("% d\n",a);
                printf("% d\n",b);
        }
        printf("% d\n",a);
        printf("% d\n",b);
        return 0;
}

Output :
10
20
10
```

```
//write a c program to automatic variable or static variable
//note : this program made by only educational purposes and this is tricky program to develop for students.

#include<stdio.h>
#include<conio.h>
void fun()
{
        int a = 10;
        static int b = 10;
        printf("%d %d\n",a,b);
        ++a;
        ++b;
}
Int main()
{
        fun();
        fun();
        fun();
        Return 0;
}

Output :
10  10
10  11
10  12
```

```
//write a c program using register variable.
#include<stdio.h>
#include<conio.h>
void main()
{
        register int a = 5;
        printf("%d",a);
        getch();
}
output :
5
```

```
//write a c program extern storage.
#include<stdio.h>
#include<conio.h>
int a = 10;
void fun()
{
        printf("%d",a);
}
int main()
{
        printf("%d\n",a);
        fun();
        if(1)
        {
                printf("%d\n",a);
        }
        return 0;
}

output :
10
10
10
```

**Q.47 what is structure ? full explanation.**

- **Structure is a user-defined name which are used to store a group of different types of values.**
- **Struct keyword is used to declare structure**
- **Structure allocates different memory for each member.**
- **The sizeof structure depends on the members of structure.**
- **Syntax :**

```
struct structure-name
{
        data-types var-name;
};
```

```
#include<stdio.h>
#include<conio.h>
struct student
{
        int roll;
        char name[20];
        double marks;
};
int main()
{
        struct student s;
        printf("enter student records:");
        scanf("%d %s %lf", &s.roll , &s.name , &s.marks);

        printf("%d %s %lf", s.roll , s.name , s.marks);

        return 0;
}

output :
enter student records :
10
amit
78.98
10 amit 79.98
```

**Q.48 what is union ? full explanation.**

- **Union is a user-defined name which is used to store a group of different types of values.**
- **Union keyword is used to declare union.**
- **Union allocates same memory for each member.**
- **The sizeof union depends on the biggest member of union.**
- **Syntax :**

```
Union union-name
{
        //member of union
};
```

```
#include<stdio.h>
#include<conio.h>
union student
{
        int roll;
        char name;
        double marks;
};
int main()
{
        union stu u;
        printf("%d",sizeof(u));
}
```

**Q.49 what is macro ? full explanation.**

- **Macro is a name which replace value , expression in place of a variable and function.**
- **Types of macro**
    1. **Simple macro**
    2. **Function macro**
- **Simple macro :**
- **Syntax :**
    **#define macro-name value**
- **Function macro :**
- **Syntax :**
    **#define macro-name(expression)**

```
//simple macro in c programming

#include<stdio.h>
#include<conio.h>
#define money 100
int main()
{
        char name[20] = "kuldip";
        char name2[20] = "amit";
        char name3[20] = "dhruven";

        printf("%s got %d rupees", name , money);
        printf("%s got %d rupees", name2 , money);
        printf("%s got %d rupees", name3 , money);

        return 0;
}
```

```
//function macro in c programming

#include<stdio.h>
#include<conio.h>
#define money()(100+10)
void main()
{
        char name[20] = "ankit";
        char name2[20] = "jay";

        printf("%s got  %d rupees\n ",name,momey());
        printf("%s got  %d rupees\n ",name2,momey());

        getch();
}
```

**Q.50 what is file handling ? full explanation.**

- **Storing data in a file and accessing data from a file called file handling.**
- **File operations :**
    1. **Create**
    2. **Read**
    3. **Write**
    4. **Delete**
    5. **Copy**
- **File operations functions are as following :**
- **open file :- fopen() , fscanf() or fgetc()**
  **fprintf() or fputc()**
  **fclose();**

```
#include<stdio.h>
#include<conio.h>
int main()
{
        FILE *fp;
        fp = fopen("c:\\users\\win10\\desktop\\amit.txt", "w");

        printf("file created successfully..!");
        fclose(fp);
        return 0;
}
```

```
#include<stdio.h>
#include<conio.h>
int main()
{
	int num;
	file *fp;
	fp = fopen("C:\\users\win10\\desktop\\amit.txt", "w");
	if(fp==null)
	{
		printf("file not created");
		exit(1);
	}
	printf("file created successfully…!\n");

	printf("enter a number:\n");
	scanf("%d",&num);

	printf("data wrote in a file");
	fprintf( fp , "%d" , num);

	fclose(fp);

	return 0;
}
```

```
#include<stdio.h>
#include<conio.h>
int main()
{
	int num;
	FILE *fp;
	if((fp=fopen("C:\\users\\win10\\desktop\\amit.txt", "r"))==null)
	{
		printf("Error Occured");
		exit(1);
	}
	fscanf(fp , "%d" , &num);
	printf("%d" , num);

	fclose(fp);
	return 0;
}
```